# A1: Computer Fundamentals - Exam Lab


This exam practice page is designed for the A1: Computer Fundamentals unit of the new IB Computer Science syllabus (first assessment 2027).

## Syllabus Links:

**A 1.1 Computer Hardware and Operation**

**A 1.2 Data representation and computer logic**

**A 1.3 Operating systems and control systems**

**A 1.4 Translation (HL only)**

"If a computer can only understand 'on' or 'off' (1 or 0), how does it manage to run a complex weather simulation and a web browser at the same time?"

## The Exam Scenario

**Scenario:** The Research Simulation Hub
A university research lab uses a high-performance cluster to run complex weather simulations. The system utilises multiple multi-core CPUs and advanced GPUs, coordinated by a custom-built operating system.

## Core Questions (HL and SL)

Give these questions a try! If you're stuck, check out the 'Strategy' section to help you get started. Once you're finished, use the Mark Scheme and Worked Example to review your progress. If these feel a bit tough right now, head back to the eBook pages to sharpen your understanding first.

## Question 1: Hardware Interaction (5 Marks)

(a) **Identify** the two main units of the central processing unit (CPU). **[2 marks]**

(b) **Outline** the role of the Program Counter (PC) register during the fetch-decode-execute cycle. **[3 marks]**

**Level 1: Strategy**

**Command Term Check:** "**Identify**" requires a specific answer from among several possibilities. "**Outline**" requires a brief account or summary.

**The Logic:** Focus on what starts the cycle. Where does the CPU know where to look next?

**Level 2: Mark Scheme**

**(a)** Award **[1]** for **Arithmetic Logic Unit (ALU)** and **[1]** for **Control Unit (CU)**.
**(b)** Award **[1]** for stating it holds the address of the next instruction. Award **[1]** for mentioning it increments after a fetch. Award **[1]** for mentioning it sends the address to the Memory Address Register (MAR).

**Level 3: Worked Example**

(a) The two main units are the Arithmetic Logic Unit and the Control Unit.

(b) The Program Counter (PC) stores the memory address of the next instruction to be processed. Once that instruction is fetched, the PC is incremented to point to the next instruction in the sequence, ensuring the fetch-decode-execute cycle continues correctly.

**Tip:** Don't confuse the PC with the Instruction Register (IR). The PC is the "map" (where we are going), the IR is the "bag" (what we are currently carrying).

## Question 2: Data & Logic (5 Marks)

A safety-critical sensor in the lab uses the following logic circuit to trigger an alarm:

Z = (A AND B) OR (NOT) C

(a) **State** the binary equivalent of the hexadecimal number **AC**. **[1 mark]**

(b) **Construct** a truth table for the Boolean expression: Z = (A AND B) OR (NOT C). **[4 marks]**

**Level 1: Strategy**

**The Logic:** For hex conversion, treat "A" and "C" as two separate 4-bit nibbles. For the truth table, there are 3 inputs (A, B, C), so there must be $2^3 = 8$ rows.

**Level 2: Mark Scheme**

**(a)** Award **[1]** for **10101100**.

**(b)** Award **[1]** for each pair of correct rows.

**Level 3: Worked Example**

(a) A = 10, C = 12. Binary: 1010 1100.

(b)

| A | B | C | A AND B | NOT C | Z (Output) |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 | 1 |

## Question 3: Operating Systems (4 Marks)

The lab's operating system manages many simultaneous sensor inputs. **Compare and contrast** the use of **polling** and **interrupt handling** for managing data from these sensors. **[4 marks]**

**Level 1: Strategy**

**Command Term Check: "Compare and contrast"** requires identifying both similarities and differences.

**The Logic:** Think about "Efficiency" vs. "Predictability." One is the CPU asking, "Are you done yet?" and the other is the device shouting, "I'm done!"

**Level 2: Mark Scheme**

Award **[1]** for a similarity (both are CPU-device communication methods).

Award **[1]** for a contrast point regarding CPU overhead (Polling has high overhead).

Award **[1]** for a contrast point regarding event predictability (Interrupts handle unpredictable events better).

Award **[1]** for a contrast point regarding power/latency.

**Level 3: Worked Example**

Both methods allow a CPU to monitor external hardware devices, but they differ in efficiency. In polling, the CPU repeatedly checks the device's status, which is predictable but wastes many clock cycles (high overhead). In contrast, interrupt handling allows the CPU to focus on other tasks until a device signals the Control Unit, making it much more efficient for systems with unpredictable inputs, though it is more complex to implement.

## HL Only: Advanced Architecture & Translation

Try to answer the questions if you need help, and have a look at the Strategy to help you get started. Once you have answered the question, you can then check the Mark Scheme and the Worked Example. If you find the questions too hard right now, then go back to the eBook pages first to develop your understanding.

### Question 4: Advanced Hardware (5 Marks)

(a) The hub's multi-core CPUs use **pipelining**. **Describe** how the **write-back stage** of pipelining improves system performance. **[3 marks]**

(b) **Explain** the design philosophy differences between a CPU and a GPU regarding **memory access** and **power efficiency**. **[2 marks]**

**Level 1: Strategy**

**Command Terms:** "**Describe**" requires a detailed account; "**Explain**" requires reasons or causes.

**The Logic:** For (a), think of the final step of an instruction. For (b), think about "General Purpose" vs. "High-Throughput Specialisation."

**Level 2: Mark Scheme**

**(a)** [1] for defining write-back (recording results to memory/registers); [1] for stating it allows the execution unit to free up for the next instruction; [1] for noting it prevents "stalls" in the pipeline.

**(b)** [1] for CPU prioritising low-latency (fast response for single tasks); [1] for GPU prioritising high-throughput (processing massive parallel data streams simultaneously).

**Level 3: Worked Example**

**(a)** The write-back stage is the final part of the pipeline where the result of an instruction is committed to memory or a register. By separating this stage, the CPU's execution unit does not have to remain idle while data is being written; it can immediately begin processing the next instruction in the pipeline, thereby increasing overall efficiency and instruction throughput.

**(b)** CPUs are designed with large caches to minimise the latency of a single thread of execution, making them "latency-oriented." GPUs, however, are "throughput-oriented," sacrificing individual thread speed for the ability to move massive amounts of data across thousands of small cores simultaneously, which is essential for simulation math.

## Question 5: Resource Management & Multitasking (6 Marks)

(a) During a simulation, the OS must manage **resource contention**. **Explain** how a **deadlock** occurs in this environment and **suggest** one way the OS might prevent it. **[4 marks]**

(b) **Distinguish** between an **open-loop** and a **closed-loop** control system within the lab's cooling infrastructure. **[2 marks]**

**Level 1: Strategy**

**The Logic:** Deadlock is a "circular wait" in which no one can move. For (b), the difference is the **feedback mechanism**.

**Level 2: Mark Scheme**

**(a)** [1] for describing two or more processes waiting for resources held by each other; [1] for identifying the "circular wait" state; [1] for mentioning it halts execution; [1] for a prevention method (e.g., resource preemption or priority scheduling).

**(b)** [1] for open-loop having no feedback (runs on a timer); [1] for closed-loop using sensors to adjust output based on actual temperature.

**Level 3: Worked Example**

**(a)** A deadlock occurs when Process A holds Resource 1 and waits for Resource 2, while simultaneously Process B holds Resource 2 and waits for Resource 1. This creates a "circular wait" where neither process can proceed. The OS can prevent this by using "Resource Preemption," which temporarily interrupts a process and forcibly takes back a resource to give it to another process, breaking the cycle.

**(b)** An open-loop system, such as a lab ventilation fan on a fixed timer, operates regardless of the actual environmental state. A closed-loop system, such as an automated server cooling unit, uses temperature sensors to provide feedback, automatically increasing fan speed when the temperature exceeds a set threshold.

## Question 6: The Translation Process (6 Marks)

The lab developers are choosing between **Just-In-Time (JIT) compilation** and a traditional **Interpreter**.

(a) **Evaluate** the use of a **bytecode interpreter** for a simulation that needs to be **cross-platform**. **[6 marks]**

**Level 1: Strategy**

**Command Term:** "**Evaluate**" means you must provide a balanced view (Pros and Cons).

**Context:** Focus on why "Cross-platform" (A1.4.1) matters here.

**Level 2: Mark Scheme**

**[1-2]** for explaining the process (Source code → Virtual Machine).

**[1-2]** for strengths (Portability: same bytecode runs on any OS with the VM; easier debugging/sandboxing).

**[1-2]** for limitations (Performance overhead: interpreting bytecode is slower than native machine code for heavy simulation math).

**Level 3: Worked Example**

A bytecode interpreter is highly effective for cross-platform development because the code is compiled into an intermediate "bytecode" that runs on a Virtual Machine (VM) rather than directly on hardware. This ensures the simulation can run on any OS without modification. However, the limitation is speed; since the VM must translate bytecode into machine code at runtime, it is less efficient than a native compiler. For high-performance simulations, this overhead might be

unacceptable, whereas a Just-In-Time (JIT) compiler might be a better compromise, as it compiles the bytecode into native code just before execution for better performance.

## How confident do you feel with this topic?

**Exam Ready:** I answered most questions correctly without looking at the eBook. I can explain these concepts to someone else and understand the "why" behind the mark scheme.

*Move on to the next topic.*

**Practitioner:** I understood the general concepts, but I missed a few marks on specific terminology or complex applications. I needed the "Strategy" guide for the harder questions.

*Annotate your answers with the Mark Scheme and revisit the eBook sections where you lost marks.You can also use the bookmarking and notes feature on the the eBook pages to highlight certain areas.*

**Novice:** I struggled to get started on the questions. I found it difficult to apply the theory to the practice scenarios.

*Go back to the eBook and Worked Examples before attempting these questions again. Perhaps use some of the Quizlets to practice the key terms and review the command terms.*